



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:

Sistemas Operativos

Asignación:

Caso de estudio: cálculo de recursos del sistema operativo para servicios en producción (Nginx + MySQL/InnoDB, instalación nativa vs. contenedores Docker)

Profesor:

Prof. Isacio Tamayo

Estudiante:

James Tack

Grupo:

1IL143

Fecha de entrega:

6 de octubre de 2026

2do Semestre 2026

De qué se trata el caso

Un cliente con poco presupuesto va a poner en producción una tienda en línea y aloja Nginx y MySQL/InnoDB en un solo servidor. El pico nocturno es de 800 a 1 200 usuarios concurrentes, con 2 a 4 conexiones cada uno y unas 400 solicitudes HTTP por segundo sobre Nginx. El caso pide dimensionar procesos, conexiones y memoria para dos servidores (el A, con 8 GB de RAM y 4 vCPU, y el B, con 4 GB y 2 vCPU) y por dos vías de despliegue, instalación nativa y contenedores Docker. Los entregables son una tabla comparativa de recursos y una justificación de escalabilidad de media página.

Además de calcular, comprobé cada resultado en una máquina virtual real. Usé vm-sistemas-operativos (Ubuntu 18.04, Docker 24.0.2 y Compose 2.18.1) y la configuré en VirtualBox con el tamaño exacto de cada escenario: 8 GB y 4 CPU para el A, 4 GB y 2 CPU para el B. En la vía nativa instalé Nginx 1.14.0 y MySQL 5.7.42 desde los repositorios de Ubuntu. En la vía Docker usé las imágenes `nginx:1.25` y `mysql:5.7` con límites de `mem_limit` y `cpus` en un `docker-compose.yml`. Cada comando lo corrí en una pestaña de Windows Terminal conectada por SSH a la VM, y las capturas son de esas pestañas.

Cómo planteé el trabajo

Partí de los datos del enunciado y de las fórmulas de la teoría. Los valores que cambian entre escenarios son estos.

Dato	Escenario A	Escenario B
RAM total / vCPUs	8 GB / 4	4 GB / 2
Overhead del SO en nativo	12,5 % de la RAM	15 % de la RAM
Overhead del SO + Docker en contenedores	25 % de la RAM	25 % de la RAM
Memoria de Nginx en nativo	0,3 GB	0,25 GB
Contenedor de Nginx (memoria / CPU)	1,0 GB / 1,5 vCPU	0,5 GB / 0,5 vCPU
max_connections de MySQL	200	100
Costo por conexión	2,5 MB	2,5 MB
Margen para buffers temporales	300 MB	250 MB

Las fórmulas son las del caso.

- **Procesos.** En nativo, `procesos_trabajadores = núcleos_físicos`; en Docker, `procesos_trabajadores = techo(cuota_cpu_contenedor)`.
- **Conexiones.** En ambas vías: `conexiones_por_proceso = (usuarios_pico x conexiones_por_usuario x margen) + procesos_trabajadores`, con 1 200 usuarios, 4 conexiones y margen de 1,5.
- **Memoria nativa.** Se calcula con `memoria_caché = RAM_total – RAM_SO – memoria_otros_servicios – (conexiones_max x costo_por_conexión)`.
- **Memoria Docker.** Se calcula con `memoria_caché = mem_limit_contenedor – (conexiones_max x costo_por_conexión)`, donde el límite de MySQL es la RAM que queda después del overhead y del contenedor de Nginx.

Tomé cuatro decisiones que el enunciado deja abiertas.

- Usé 1 GB = 1 024 MB, que es como Linux y Docker cuentan la memoria.
- El margen de 300 MB (A) y 250 MB (B) lo resto después de la fórmula, porque el texto dice que al remanente se le descuentan las conexiones y los buffers temporales. La `memoria_caché` de la fórmula es el techo teórico, y la cifra que configuro en `innodb_buffer_pool_size` es esa cifra menos el margen.
- MySQL 5.7 redondea hacia arriba el buffer pool a un múltiplo de `innodb_buffer_pool_chunk_size` por las instancias, que por defecto es 128 MB. Con eso, 6 060 MB habrían pasado a 6 144 y se habría gastado el margen. Fijé `innodb_buffer_pool_chunk_size = 4M` y una sola instancia, y redondeé hacia abajo a múltiplos de 4 MB (6 060, 4 320, 2 724 y 2 060 MB).
- Puse `worker_rlimit_nofile` en 16 384, por encima de `worker_connections`, para que Nginx no se quede sin descriptores de archivo antes de llegar a su límite de conexiones.

Memoria de cálculo

Escenario A: 8 GB y 4 vCPU

```
NATIVO
RAM_SO               = 8 GB x 12,5 %              = 1,000 GB
Nginx                 = 0,300 GB
conexiones            = 200 x 2,5 MB = 500 MB      = 0,488 GB
memoria_caché         = 8 - 1,000 - 0,300 - 0,488  = 6,211 GB (6 360,8 MB)
buffer pool           = 6 360,8 MB - 300 MB        = 6 060,8 MB (5,92 GB)
procesos_trabajadores = 4 núcleos
conexiones_por_proceso= 1 200 x 4 x 1,5 + 4        = 7 204

DOCKER
overhead              = 8 GB x 25 %                = 2,000 GB
RAM libre             = 8 - 2                      = 6,000 GB
mem_limit Nginx       = 1,0 GB con 1,5 vCPU
mem_limit MySQL       = 6,0 - 1,0                  = 5,0 GB con 4 - 1,5 = 2,5 vCPU
memoria_caché         = 5 120 MB - 500 MB          = 4 620 MB (4,51 GB)
buffer pool           = 4 620 MB - 300 MB          = 4 320 MB (4,22 GB)
procesos_trabajadores = techo(1,5)                 = 2
conexiones_por_proceso= 1 200 x 4 x 1,5 + 2        = 7 202
```

Escenario B: 4 GB y 2 vCPU

```
NATIVO
RAM_SO               = 4 GB x 15 %                = 0,600 GB
Nginx                 = 0,250 GB
conexiones            = 100 x 2,5 MB = 250 MB      = 0,244 GB
memoria_caché         = 4 - 0,600 - 0,250 - 0,244  = 2,906 GB (2 975,6 MB)
buffer pool           = 2 975,6 MB - 250 MB        = 2 725,6 MB (2,66 GB)
procesos_trabajadores = 2 núcleos
conexiones_por_proceso= 1 200 x 4 x 1,5 + 2        = 7 202

DOCKER
overhead              = 4 GB x 25 %                = 1,000 GB
RAM libre             = 4 - 1                      = 3,000 GB
```

```
mem_limit Nginx       = 0,5 GB con 0,5 vCPU
mem_limit MySQL       = 3,0 - 0,5                  = 2,5 GB con 2 - 0,5 = 1,5 vCPU
memoria_caché         = 2 560 MB - 250 MB          = 2 310 MB (2,26 GB)
buffer pool           = 2 310 MB - 250 MB          = 2 060 MB (2,01 GB)
procesos_trabajadores = techo(0,5)                 = 1
conexiones_por_proceso= 1 200 x 4 x 1,5 + 1        = 7 201
```

Preparación del entorno

Paso 1. Encender la máquina virtual

La VM estaba apagada con 2 GB y 2 CPU. Antes de arrancarla la dejé con 8 192 MB y 4 CPU con `VBoxManage modifyvm`, que son los datos del Escenario A, y la encendí en modo headless. `showvminfo` confirma `memory=8192`, `cpus=4` y `VMState="running"`. El Vagrantfile fija 2 GB, así que no usé `vagrant up`, que habría deshecho el cambio.

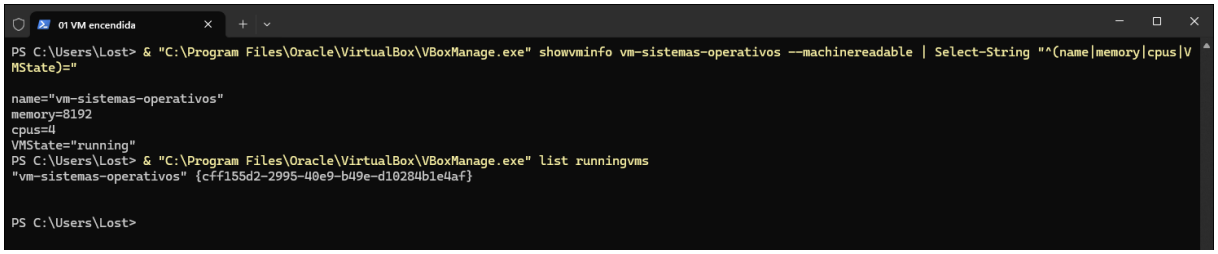


Figura 1. VBoxManage confirma que vm-sistemas-operativos está encendida con 8 192 MB y 4 CPU.

Paso 2. Copiar los archivos a la VM

Preparé en Windows la carpeta `src` con `calc.py` (el cálculo de las dos tablas) y una carpeta por escenario, `A` y `B`, cada una con `nativo` y `docker`. Cada subcarpeta tiene sus archivos de configuración. La copié a `~/casoestudio` con `scp` y la llave privada que genera Vagrant.

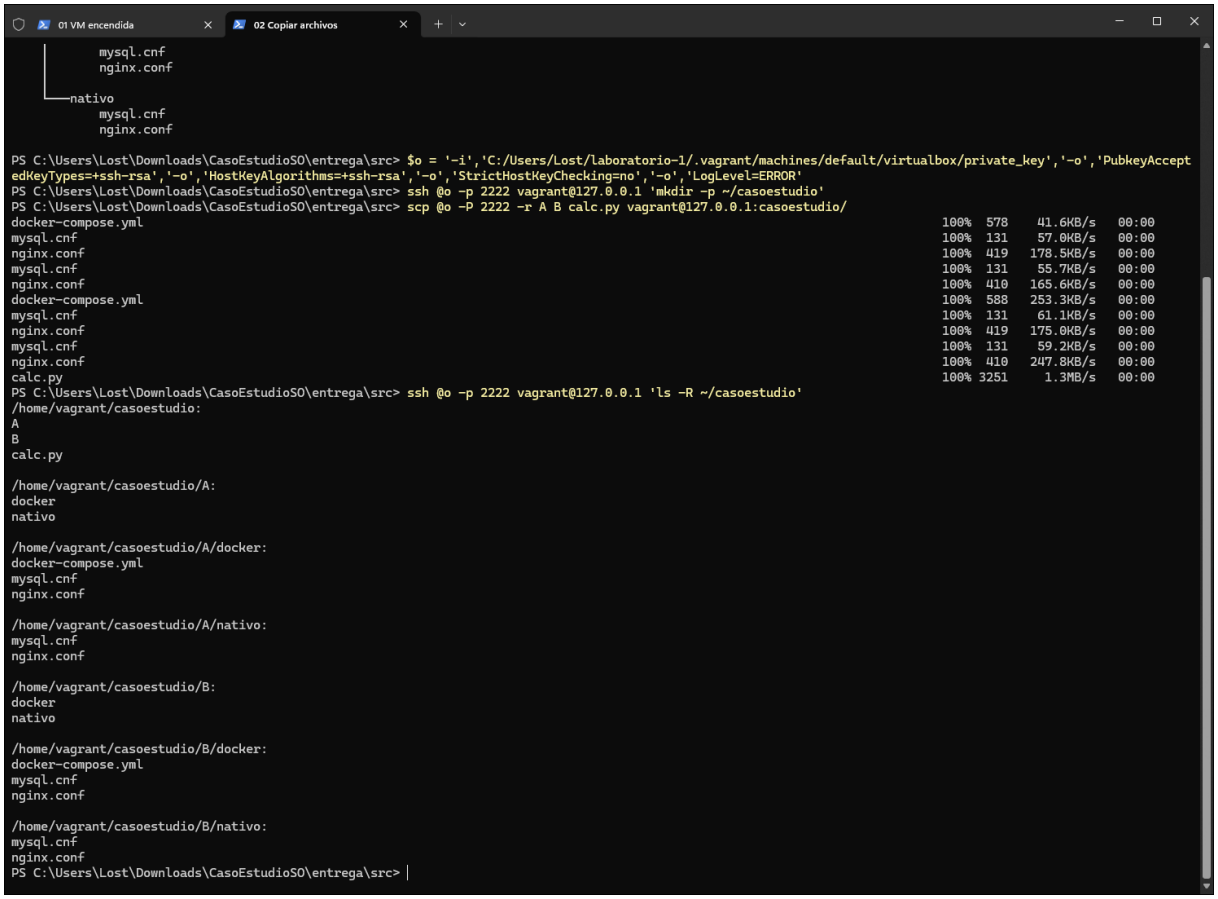


Figura 2. Árbol de archivos en Windows, copia con scp y listado en la VM.

Paso 3. Revisar el entorno dentro de la VM

Dentro de la VM verifiqué que `nproc` devuelve 4 y que `free -h` muestra 7,8 GB (el kernel y el firmware se quedan con una parte de los 8 192 MB). Hay 2 GB de swap activos, `vm.swappiness` está en 60 y `ulimit -n` en 1 024. Nginx, MySQL y `apache2-utils` no están instalados. También hay contenedores de laboratorios anteriores, detenidos, que no estorban.

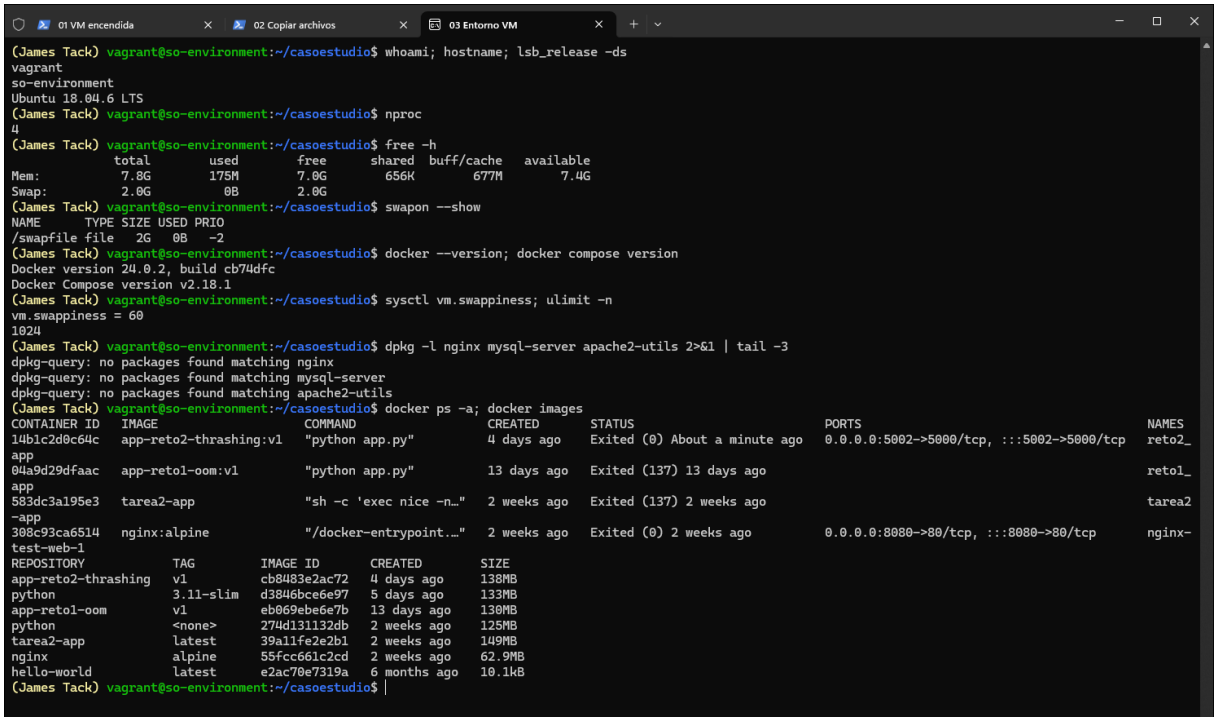


Figura 3. Cuatro CPU, 7,8 GB de RAM, Docker 24.0.2 y Compose 2.18.1; Nginx y MySQL todavía sin instalar.

Paso 4. Calcular con calc.py

`calc.py` aplica las fórmulas del caso a los dos escenarios y por las dos vías. Lo escribí con las constantes del enunciado arriba (usuarios, conexiones, margen, costo por conexión) y un diccionario por escenario, de modo que cambiar un dato recalcula todo. La salida coincide con la memoria de cálculo de este informe, y es la fuente de la tabla del Entregable 1.

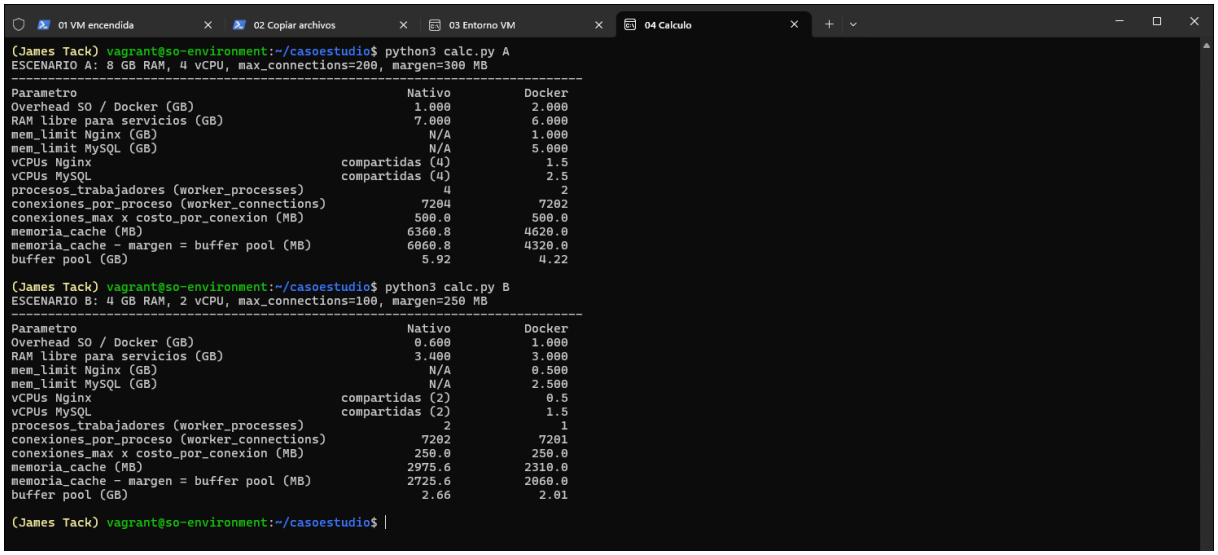


Figura 4. Salida de calc.py para los escenarios A y B.

Paso 5. Instalar Nginx, MySQL y ApacheBench

Para la vía nativa instalé `nginx`, `mysql-server` y `apache2-utils` con `apt-get`. ApacheBench (`ab`) sirve para la prueba de carga de Nginx. Las versiones son Nginx 1.14.0, MySQL 5.7.42 y ab 2.3, y los dos servicios quedan activos con su configuración por defecto.

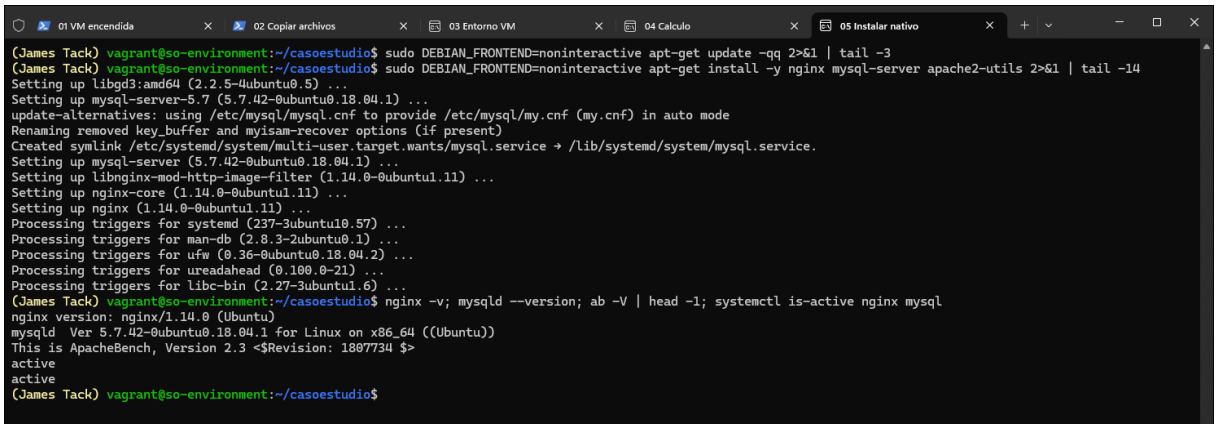


Figura 5. Instalación de los paquetes y servicios nginx y mysql activos.

Escenario A (8 GB y 4 vCPU)

Vía 1. Instalación nativa

Paso 6. Configurar Nginx

Guardé el `nginx.conf` original como `nginx.conf.orig` y copié el mío. Trae `worker_processes 4` (los cuatro núcleos), `worker_connections 7204` (la fórmula de conexiones) y `worker_rlimit_nofile 16384`. `nginx -t` da la sintaxis correcta y el servicio se reinicia sin problemas.

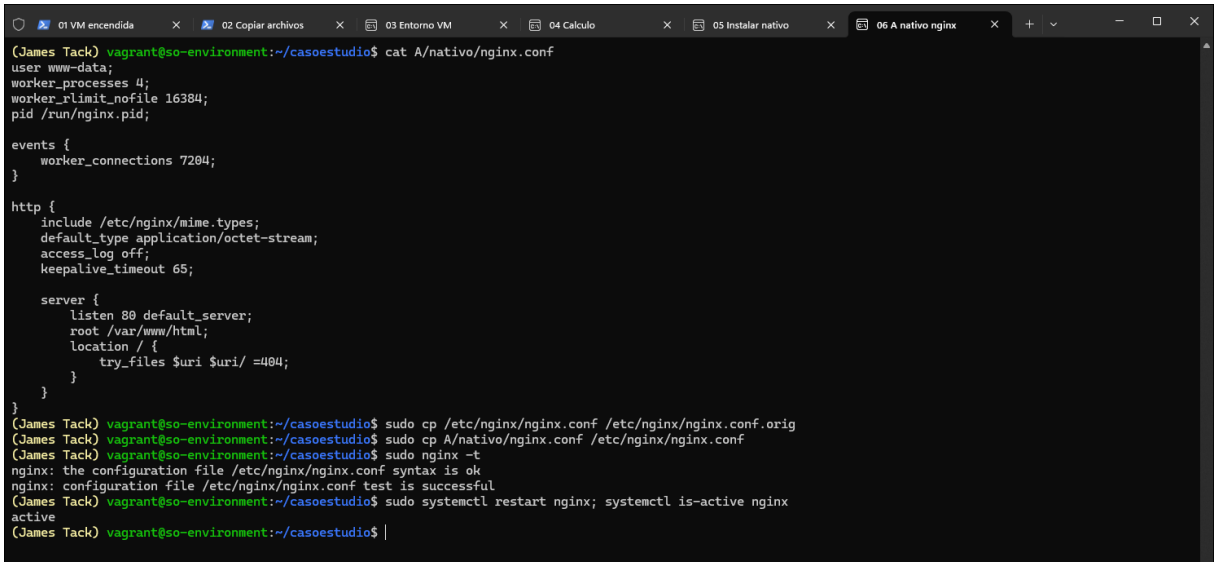


Figura 6. nginx.conf del Escenario A nativo, prueba de sintaxis y reinicio.

Paso 7. Configurar MySQL

Copié `mysql.cnf` a `/etc/mysql/mysql.conf.d/zz-caso.cnf`, que MySQL lee al final y por eso pisa los valores por defecto. Define `max_connections = 200` y `innodb_buffer_pool_size = 6060M`, con chunks de 4 MB y una instancia. El servicio reinició y quedó activo.

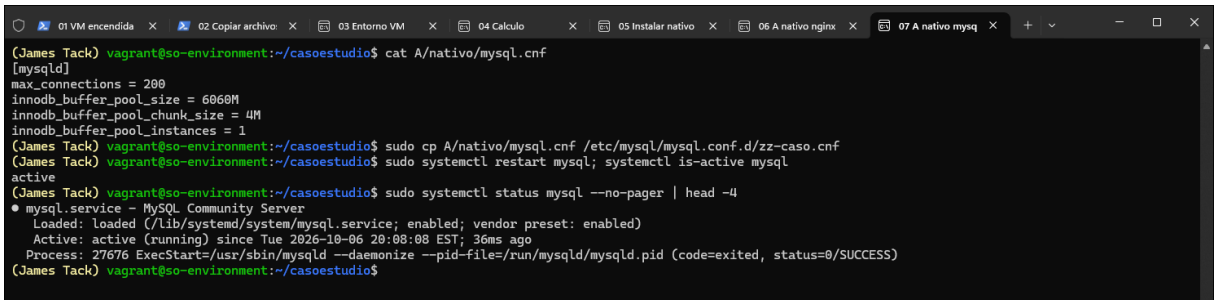


Figura 7. mysql.cnf del Escenario A nativo y reinicio de MySQL.

Paso 8. Verificar Nginx

`nginx -T` muestra la configuración que Nginx cargó de verdad: 4 procesos, 7 204 conexiones y 16 384 descriptores. `ps` lista un proceso maestro y cuatro workers, y `/proc/<pid>/limits` confirma `Max open files 16384`.

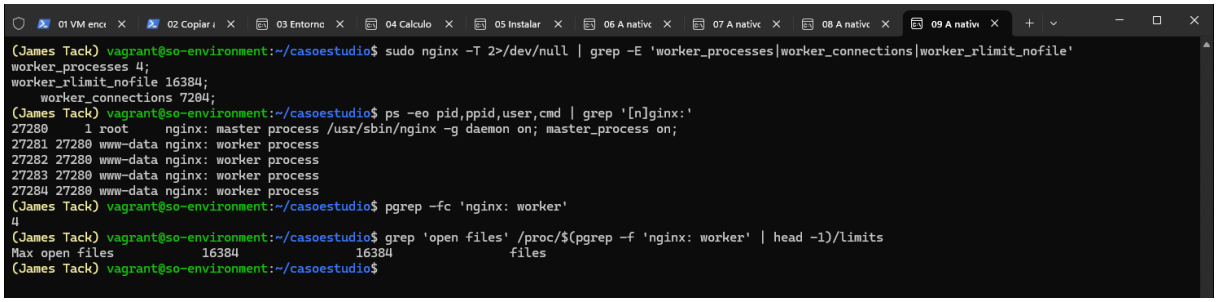


Figura 8. Cuatro workers de Nginx y límite de 16 384 descriptores de archivo.

Paso 9. Verificar MySQL

Consulté las variables desde MySQL. `max_connections` vale 200 y `innodb_buffer_pool_size` vale 6 354 370 560 bytes, o sea 6 060 MB (5,918 GB), con chunk de 4 MB. El proceso `mysqld` tiene 8,0 GB de memoria virtual pero solo 666 MB residentes, porque el buffer pool se reserva entero y se llena a medida que se leen páginas. La RAM usada del servidor entero es de 859 MB.

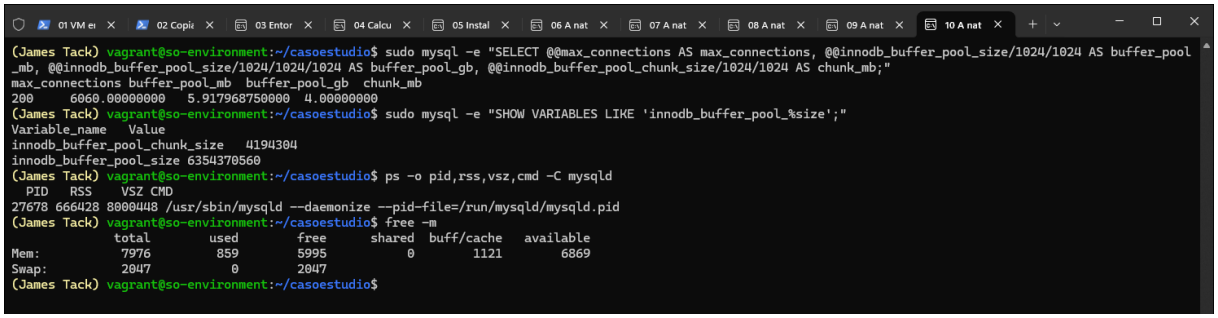


Figura 9. MySQL con buffer pool de 6 060 MB y 200 conexiones máximas; mysqld con 666 MB residentes.

Paso 10. Prueba de carga

Lancé `ab -k -n 20000 -c 400` contra Nginx, con 400 conexiones concurrentes con Keep-Alive. Terminó con 20 000 solicitudes completas y 0 fallidas, a 52 337 por segundo y 7,6 ms por solicitud en promedio. La RAM usada quedó en 852 MB, `dmesg` no registra ningún evento de OOM y los dos servicios siguen activos. El cliente corre en la misma VM y por loopback, así que el número es una referencia de capacidad y no una simulación de las 400 solicitudes por segundo reales.

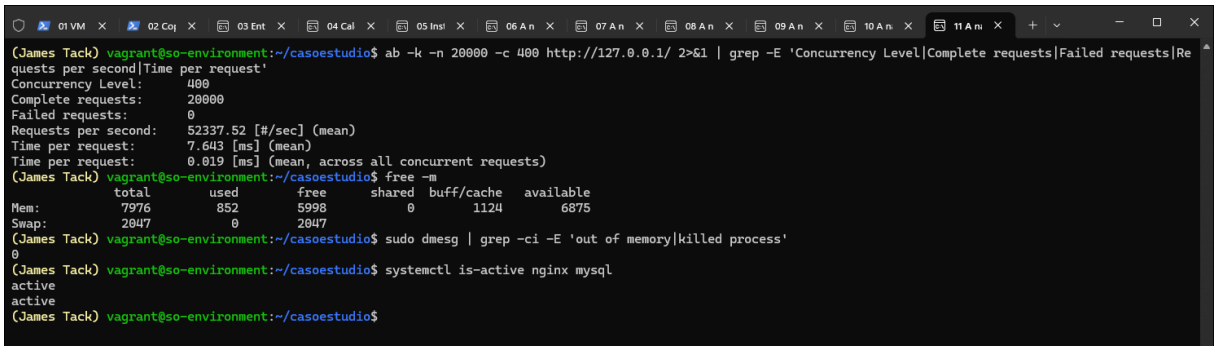


Figura 10. ab con 400 conexiones concurrentes: 0 fallidas, sin OOM y servicios activos.

Vía 2. Contenedores Docker

Paso 11. Preparar el despliegue

Detuve Nginx y MySQL nativos para liberar los puertos 80 y 3306 (`ss` no encuentra nada escuchando) y pasé a la carpeta `A/docker`. El `docker-compose.yml` define dos servicios. Nginx lleva `mem_limit: 1g`, `cpus: 1.5` y `ulimits nofile 32768`. MySQL lleva `mem_limit: 5g`, `cpus: 2.5` y `ulimits nofile 65535`. En los dos puse `memswap_limit` igual a `mem_limit`, de modo que el contenedor no tiene swap.

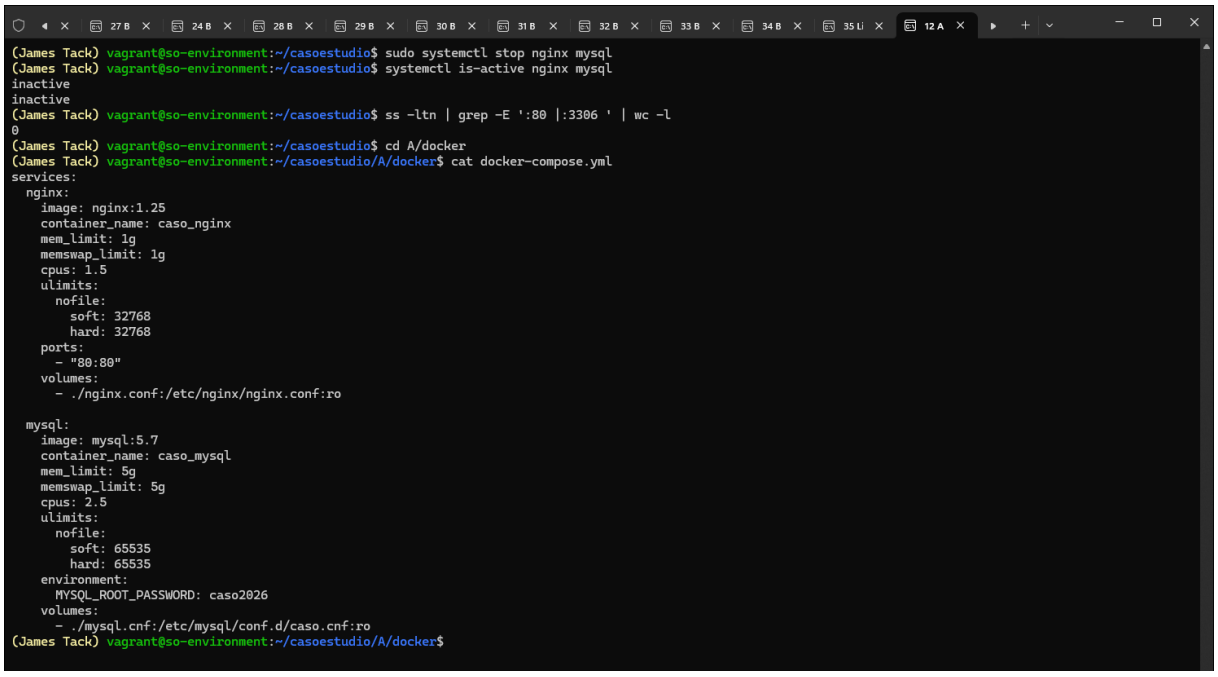


Figura 11. Servicios nativos detenidos, puertos libres y docker-compose.yml del Escenario A.

La configuración de los servicios va montada como solo lectura. Nginx lleva 2 workers (`techo(1,5)`) y 7 202 conexiones, y MySQL un buffer pool de 4 320 MB con 200 conexiones.

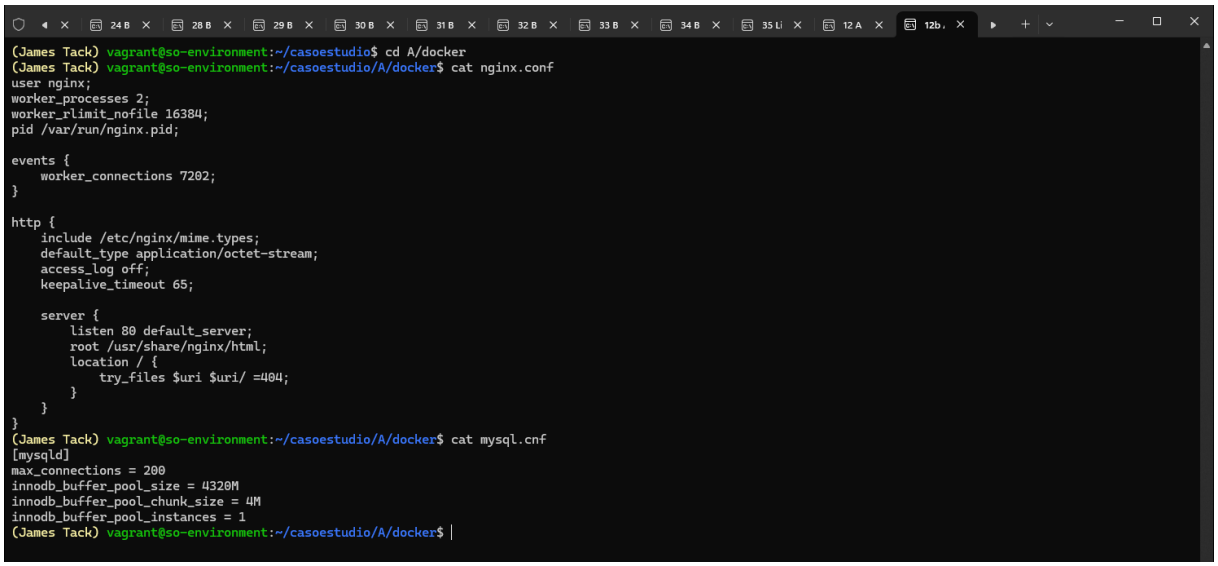


Figura 12. nginx.conf y mysql.cnf del Escenario A en Docker.

Paso 12. Levantar los contenedores

`docker compose pull` bajó la imagen de MySQL (la de Nginx ya estaba) y `docker compose up -d` creó y arrancó los dos contenedores.

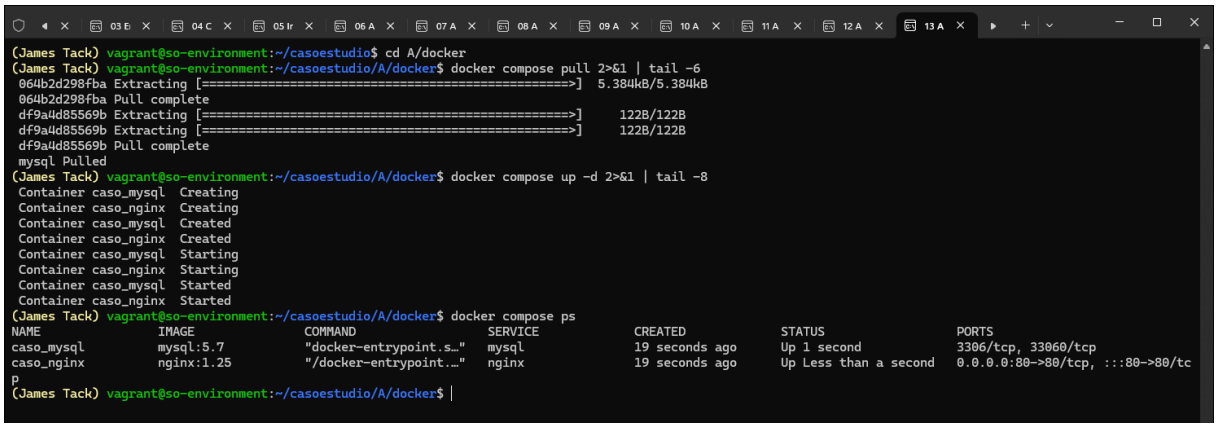


Figura 13. Descarga de imágenes y arranque de caso_nginx y caso_mysql.

Paso 13. Verificar los límites

Cruzé tres fuentes. `docker inspect` dice `Memory=1073741824` para Nginx y `5368709120` para MySQL (1 GiB y 5 GiB), con `MemorySwap` igual y `NanoCpus` de 1 500 000 000 y 2 500 000 000. Dentro de cada contenedor, el cgroup v2 lo confirma con `memory.max`, `memory.swap.max = 0` y `cpu.max` de `150000 100000` y `250000 100000`, que son 1,5 y 2,5 CPU. `docker stats` muestra los mismos topes, 1 GiB y 5 GiB.

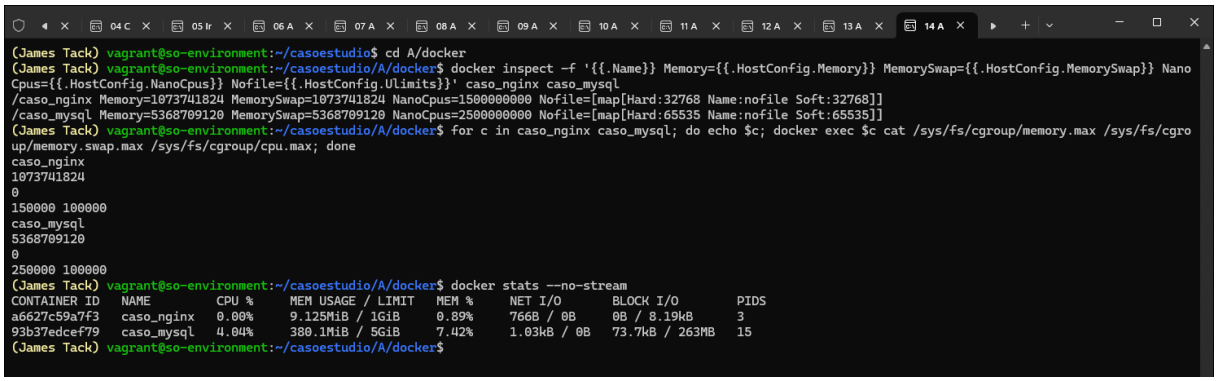


Figura 14. Límites de memoria, swap y CPU de los dos contenedores del Escenario A.

Paso 14. Verificar Nginx en el contenedor

`nginx -T` dentro del contenedor muestra `worker_processes 2`, `worker_connections 7202` y `worker_rlimit_nofile 16384`. `docker top` lista el maestro y dos workers, y el límite de archivos abiertos es 32 768, el que fijé en `ulimits`.

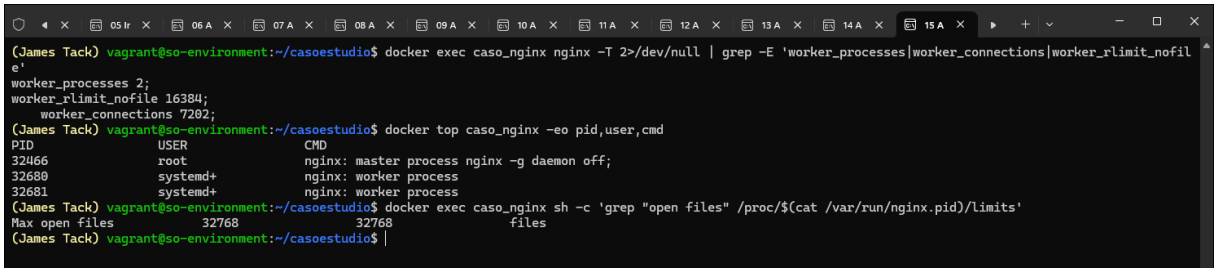


Figura 15. Dos workers de Nginx y 7 202 conexiones por proceso en el contenedor.

Paso 15. Verificar MySQL en el contenedor

Esperé a que MySQL aceptara consultas, porque en el primer arranque inicializa los datos. `max_connections` es 200 y el buffer pool es de 4 320 MB (4,219 GB). `docker stats` marca 514 MiB de 5 GiB, y el servidor usa 775 MB en total.

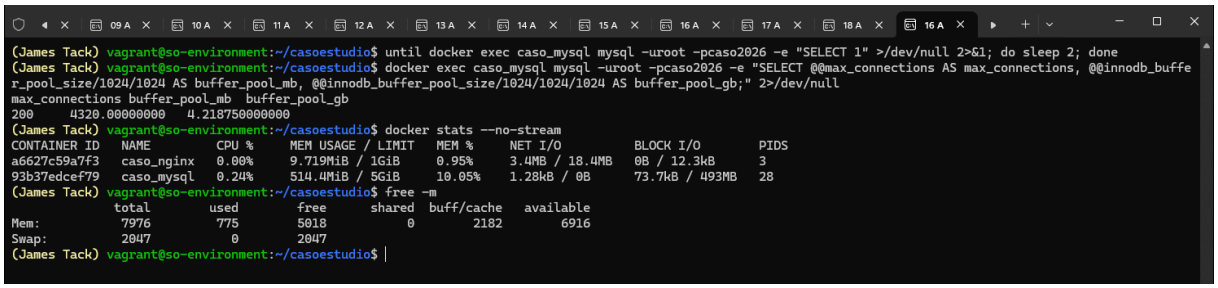


Figura 16. MySQL en Docker con buffer pool de 4 320 MB, 200 conexiones y 514 MiB usados de 5 GiB.

Paso 16. Prueba de carga

La misma prueba de `ab` contra el Nginx del contenedor terminó con 0 fallidas, a 27 938 por segundo y 14,3 ms por solicitud. Con la cuota de 1,5 CPU el rendimiento es casi la mitad que en nativo (52 337 por segundo), algo esperable. Ningún contenedor fue terminado por el OOM Killer (`OOMKilled=false`, `Restarts=0`) y `dmesg` está limpio.



Figura 17. ab contra Nginx en Docker: 0 fallidas; OOMKilled=false en ambos contenedores.

Paso 17. Medir el overhead real de Docker

El caso reserva el 25 % de la RAM para el daemon, `containerd` y `overlay2`, o sea 2 GB. Medí lo que ocupan de verdad los procesos de Docker. `dockerd` tiene 97 MB residentes, `containerd` 46 MB y los dos shims unos 19 MB, en total 159 MB. Sin embargo, el 25 % es una reserva prudente de presupuesto (cubre también la caché de las capas de `overlay2` y el kernel), no una medición del daemon. El driver es `overlay2` y el cgroup es el v2 con driver `systemd`.



Figura 18. Memoria residente de dockerd, containerd y shims: 159 MB.

Paso 18. Bajar el escenario A

`docker compose down` elimina los dos contenedores y la red. `docker ps -a` muestra que solo quedan los contenedores de laboratorios anteriores y `free -m` baja a 249 MB usados.

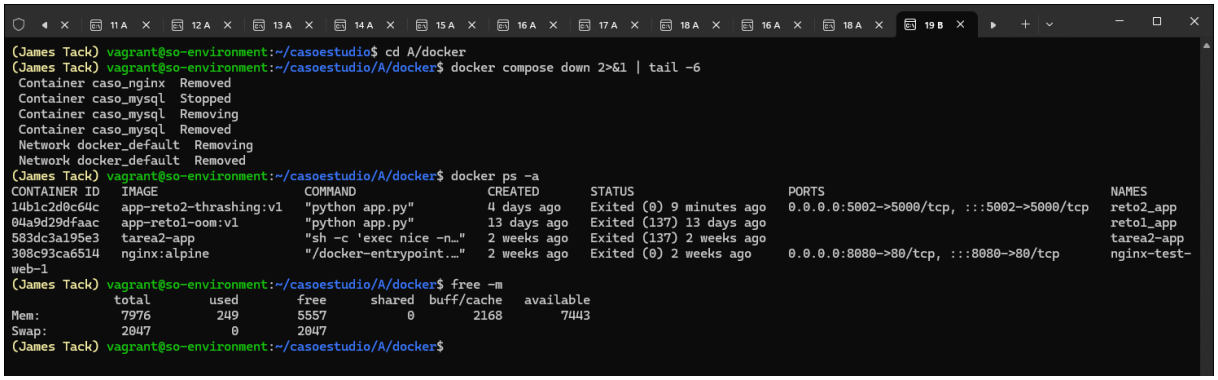


Figura 19. Contenedores del Escenario A eliminados.

Escenario B (4 GB y 2 vCPU)

Cambiar la VM al tamaño del escenario B

Apagué la VM con `poweroff`, esperé a que VirtualBox la diera por detenida y la reconfiguré con `modifyvm --memory 4096 --cpus 2`. Al volver a encenderla, `showvminfo` marca `memory=4096` y `cpus=2`.

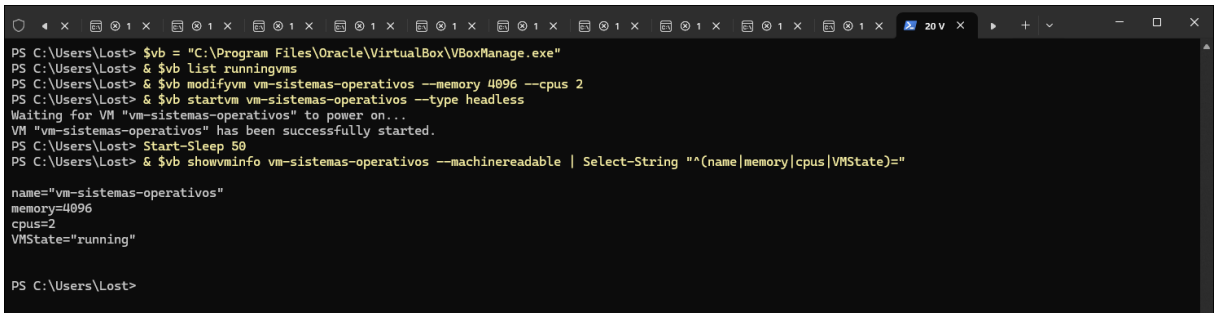


Figura 20. VM reconfigurada a 4 096 MB y 2 CPU.

Dentro de la VM, `nproc` devuelve 2 y `free -h` muestra 3,9 GB. Como el apagado no tocó el disco, los paquetes siguen instalados y los servicios nativos arrancaron solos con la configuración del Escenario A, que reemplazo en los pasos siguientes.

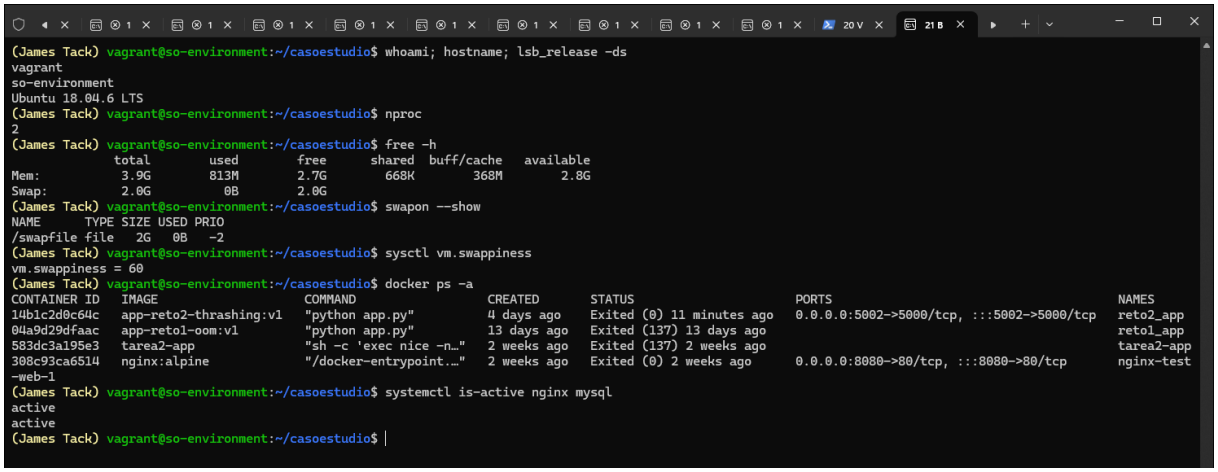


Figura 21. La VM con 2 CPU y 3,9 GB de RAM.

Vía 1. Instalación nativa

Paso 19. Configurar Nginx

El `nginx.conf` del B pone `worker_processes 2` y `worker_connections 7202`. Lo copié sobre el anterior, `nginx -t` pasó y reinicié el servicio.

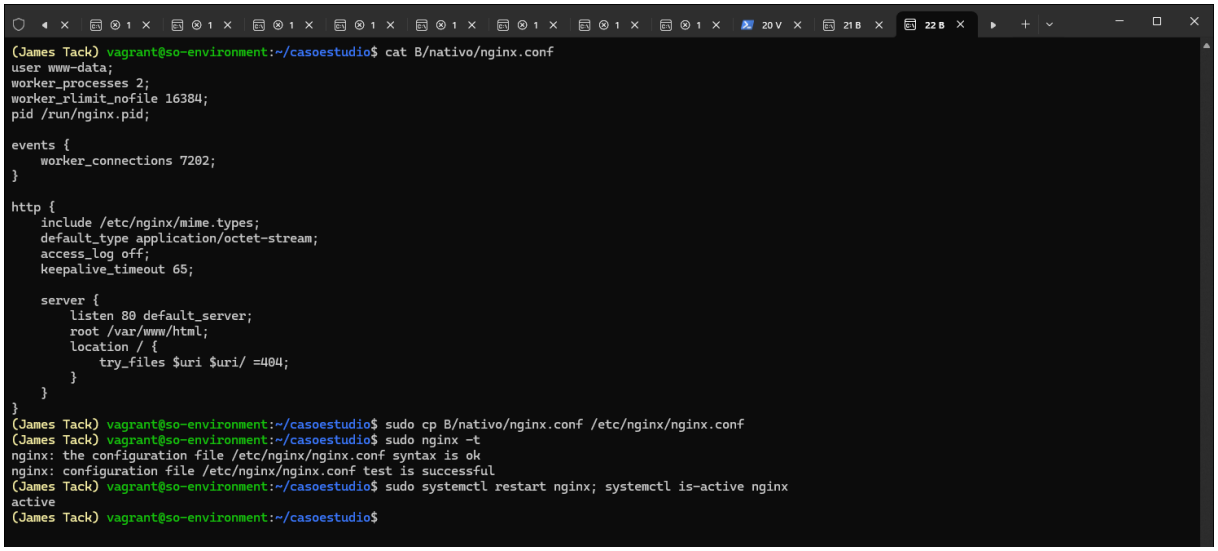


Figura 22. nginx.conf del Escenario B nativo.

Paso 20. Configurar MySQL

El `mysql.cnf` del B fija `max_connections = 100` e `innodb_buffer_pool_size = 2724M`. Lo copié a `zz-caso.cnf` y reinicié MySQL.

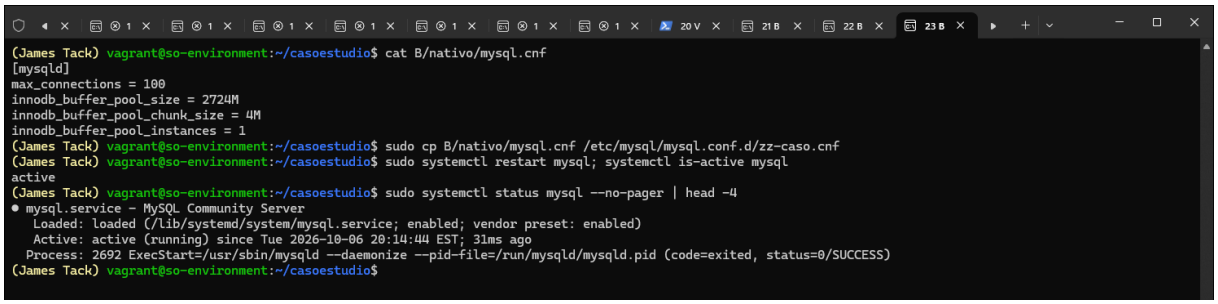


Figura 23. mysql.cnf del Escenario B nativo y reinicio de MySQL.

Paso 21. Política de swap en nativo

En la vía nativa el kernel decide cuánto swap usar, y el valor por defecto de `vm.swappiness` es 60, pensado para escritorios. En un servidor de base de datos conviene que el kernel descarte caché de archivos antes de mandar el buffer pool al disco. Lo bajé a 1 con `sysctl` y lo guardé en `/etc/sysctl.d/99-caso.conf` para que sobreviva a un reinicio. El swap de 2 GB sigue disponible como red de seguridad. El ajuste es del servidor y vale igual para el escenario A, pero lo apliqué recién aquí. La corrida nativa del A se midió entonces con el valor por defecto, aunque el swap en uso fue de 0 B en las dos.

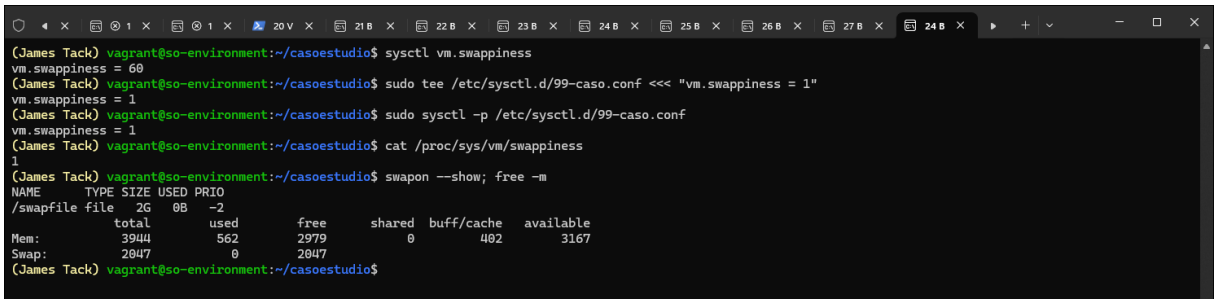


Figura 24. vm.swappiness pasa de 60 a 1 y queda guardado en sysctl.d.

Paso 22. Verificar Nginx

Nginx tiene un maestro y 2 workers, con 7 202 conexiones por proceso y 16 384 descriptores de archivo.

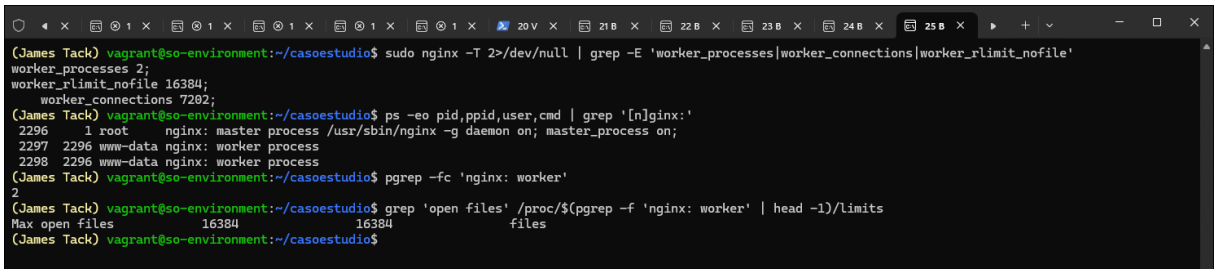


Figura 25. Dos workers de Nginx en el Escenario B nativo.

Paso 23. Verificar MySQL

MySQL muestra `max_connections = 100` y un buffer pool de 2 856 321 024 bytes, o sea 2 724 MB (2,66 GB). El proceso tiene 4,3 GB de memoria virtual y 393 MB residentes, y el servidor usa 537 MB en total.

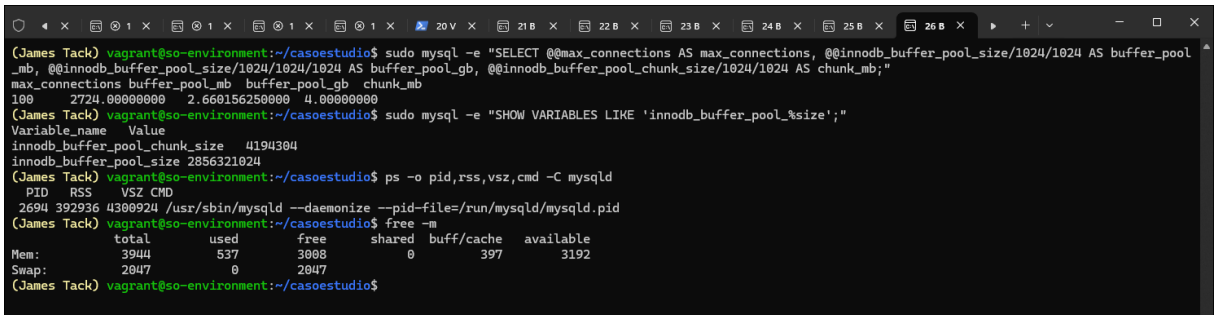


Figura 26. MySQL del Escenario B con buffer pool de 2 724 MB y 100 conexiones máximas.

Paso 24. Prueba de carga

Con 400 conexiones concurrentes, `ab` completó las 20 000 solicitudes sin fallos, a 33 023 por segundo y 12,1 ms en promedio. La RAM usada fue de 541 MB, no hay eventos de OOM y los servicios siguen activos.

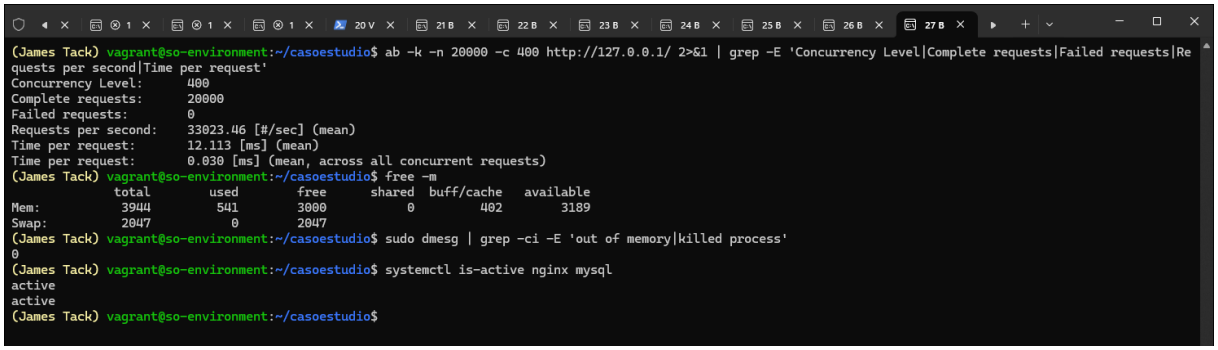


Figura 27. ab en el Escenario B nativo: 0 fallidas y sin OOM.

Vía 2. Contenedores Docker

Paso 25. Preparar el despliegue

Detuve los servicios nativos y comprobé que los puertos 80 y 3306 quedaban libres. El `docker-compose.yml` del B lleva Nginx con `mem_limit: 512m` y `cpus: 0.5`, y MySQL con `mem_limit: 2560m` (2,5 GB) y `cpus: 1.5`. Igual que en el A, `memswap_limit` es igual a `mem_limit`.

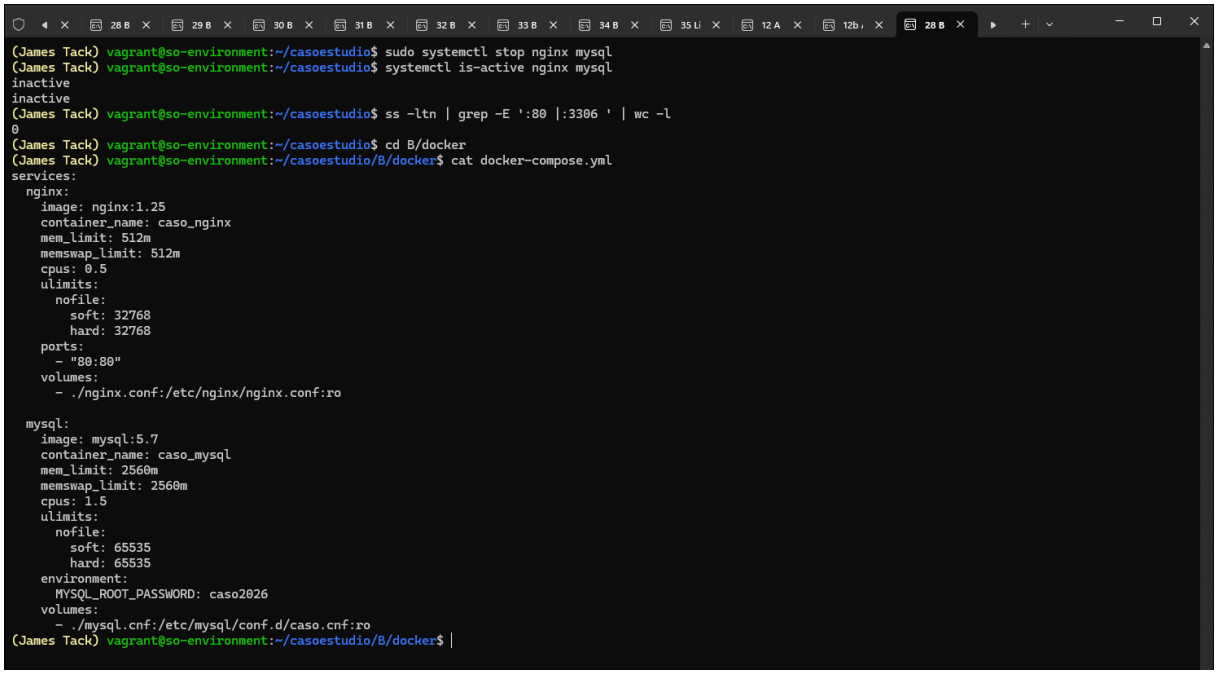


Figura 28. Servicios nativos detenidos y docker-compose.yml del Escenario B.

Nginx lleva un solo worker (`techo(0,5)`) y 7 201 conexiones. MySQL lleva un buffer pool de 2 060 MB y 100 conexiones.

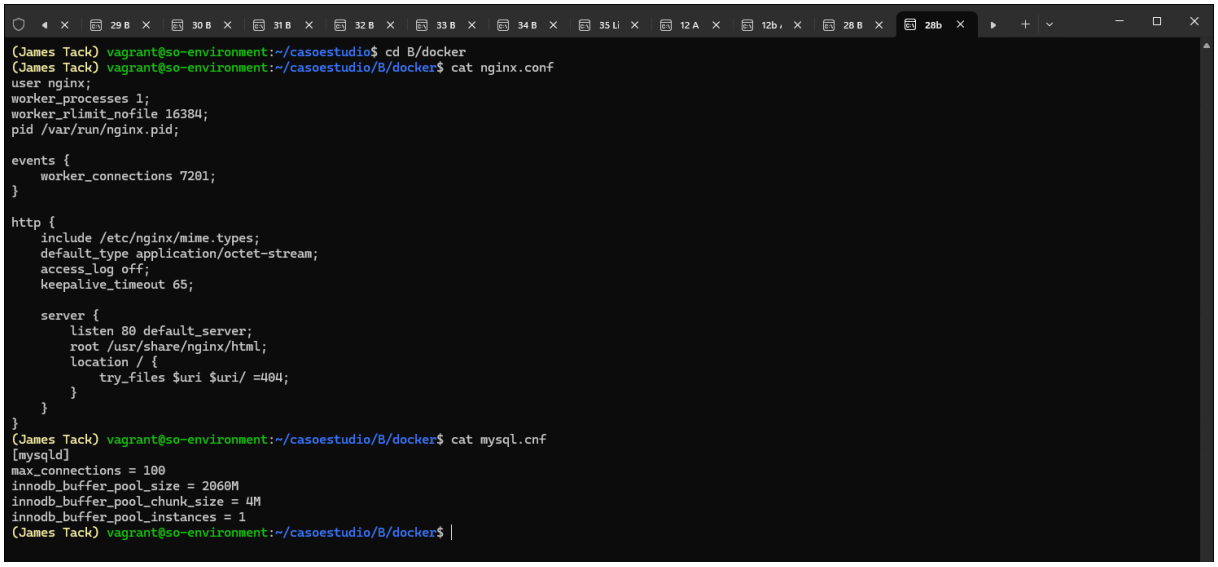


Figura 29. nginx.conf y mysql.cnf del Escenario B en Docker.

Paso 26. Levantar los contenedores

Con las imágenes ya descargadas, `docker compose up -d` creó los dos contenedores en pocos segundos.

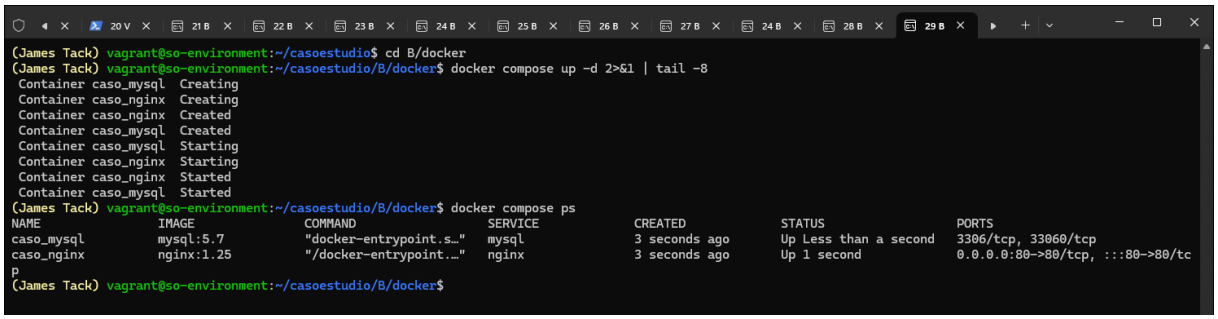


Figura 30. Arranque de caso_nginx y caso_mysql en el Escenario B.

Paso 27. Verificar los límites

`docker inspect` da `Memory=536870912` (512 MiB) y `NanoCpus=500000000` para Nginx, y `Memory=2684354560` (2,5 GiB) y `NanoCpus=1500000000` para MySQL, con `MemorySwap` igual a `Memory` en ambos. En el cgroup, `cpu.max` es `50000 100000` (0,5 CPU) y `150000 100000` (1,5 CPU), y `memory.swap.max` es 0.

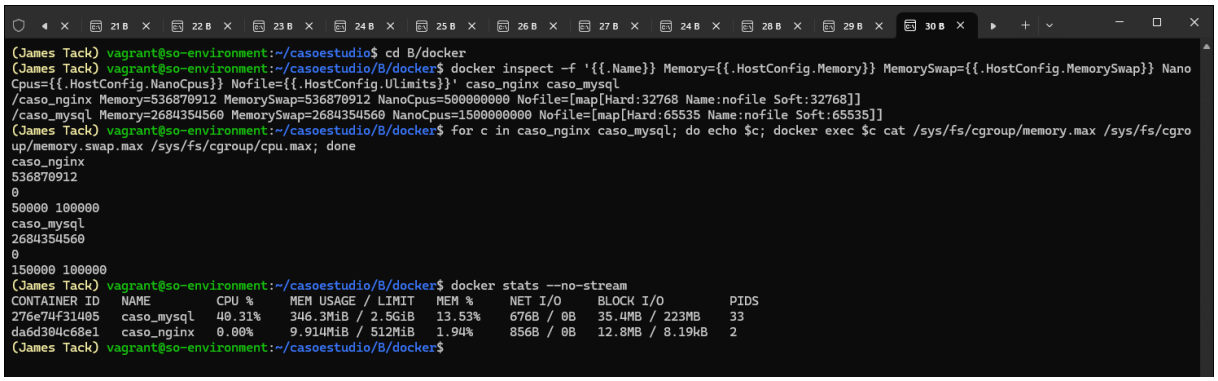


Figura 31. Límites de memoria, swap y CPU de los contenedores del Escenario B.

Paso 28. Verificar Nginx en el contenedor

El contenedor tiene un maestro y un solo worker, con 7 201 conexiones por proceso y 32 768 descriptores de archivo.

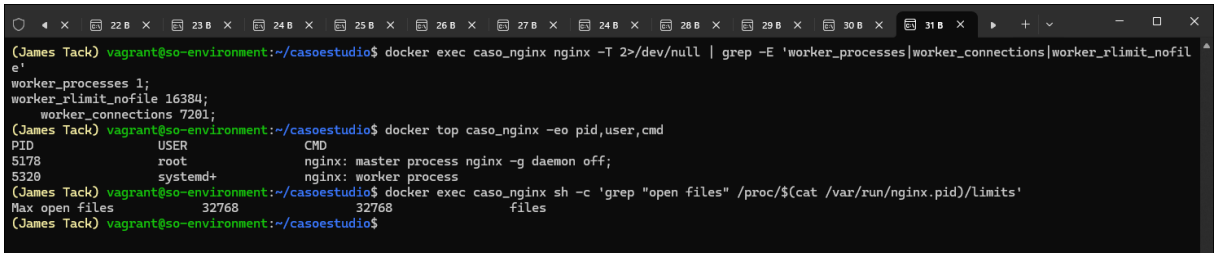


Figura 32. Un worker de Nginx con cuota de 0,5 CPU.

Paso 29. Verificar MySQL en el contenedor

MySQL responde con `max_connections = 100` y un buffer pool de 2 060 MB (2,012 GB). `docker stats` marca 359 MiB de 2,5 GiB.

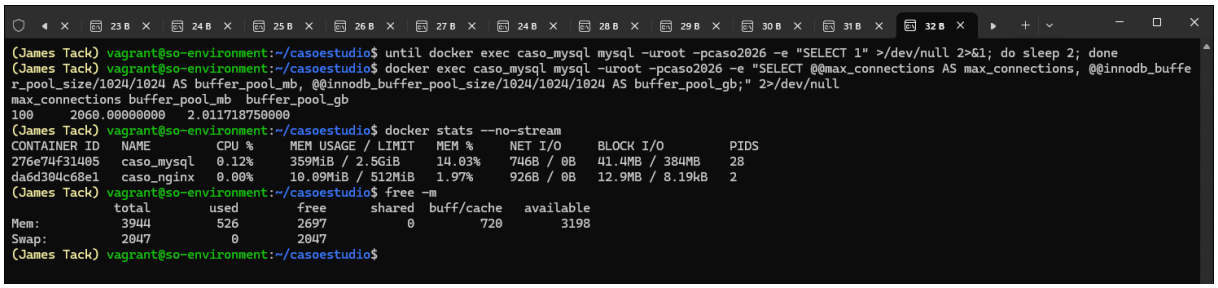


Figura 33. MySQL en Docker con buffer pool de 2 060 MB y 359 MiB usados de 2,5 GiB.

Paso 30. Prueba de carga

Con la cuota de 0,5 CPU, `ab` completó las 20 000 solicitudes sin fallos, pero a 9 714 por segundo y 41,2 ms en promedio, contra 33 023 y 12,1 ms en nativo. Es el costo de que Nginx solo pueda usar media CPU. Ningún contenedor fue terminado por el OOM Killer.

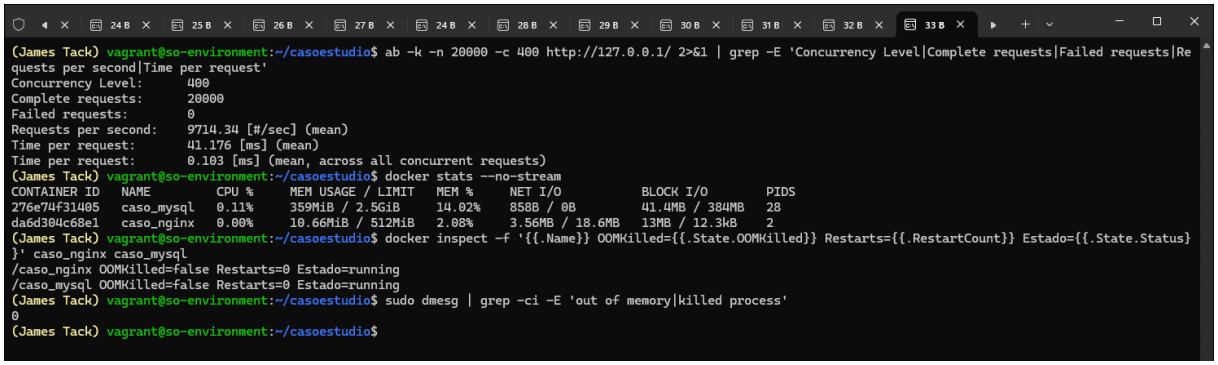


Figura 34. ab contra Nginx limitado a 0,5 CPU: 0 fallidas, pero con menor rendimiento.

Paso 31. Medir el overhead real de Docker

En el B, el daemon y `containerd` ocupan 144 MB (`dockerd` 83 MB, `containerd` 44 MB y los shims 19 MB), frente a 1 GB reservado en el cálculo.

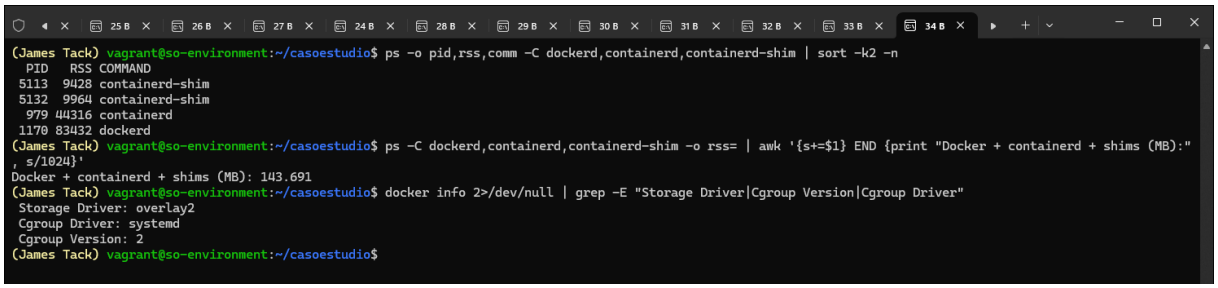


Figura 35. Memoria residente de dockerd, containerd y shims en el Escenario B: 144 MB.

Limpieza

Bajé los contenedores del B con `docker compose down` y dejé Nginx y MySQL nativos detenidos y deshabilitados (`systemctl disable`), para que no ocupen los puertos 80 y 3306 ni arranquen solos en otros laboratorios. Los puertos quedan libres.

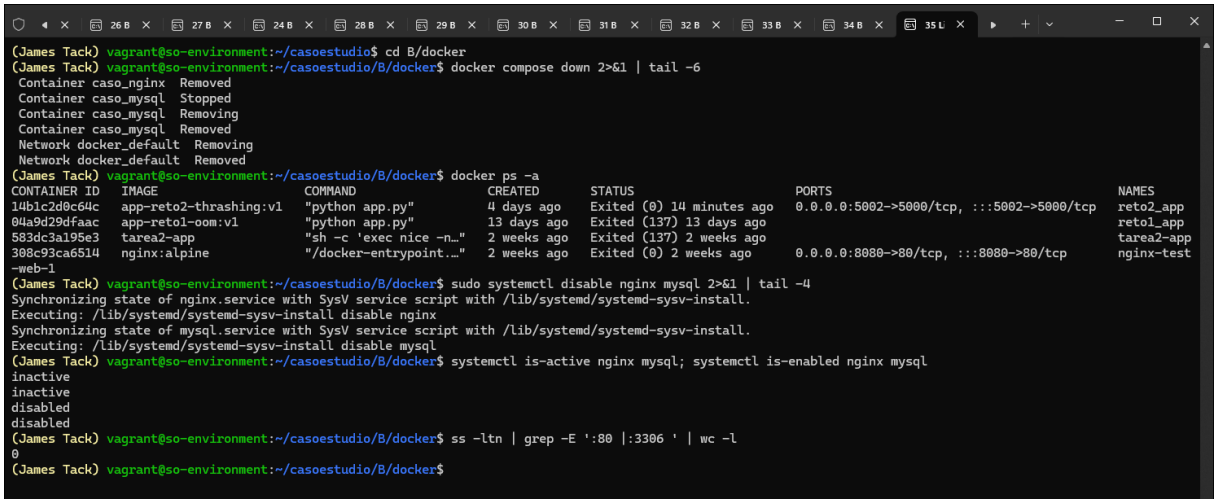


Figura 36. Contenedores del caso eliminados y servicios nativos detenidos y deshabilitados.

Entregable 1: tabla comparativa de recursos

Parámetro / Recurso	A (8 GB / 4 vCPU) Nativo	A (8 GB / 4 vCPU) Docker	B (4 GB / 2 vCPU) Nativo	B (4 GB / 2 vCPU) Docker
Overhead SO / Docker Daemon (GB)	1,0	2,0	0,6	1,0
RAM libre para servicios (GB)	7,0	6,0	3,4	3,0
Límite RAM contenedor Nginx (mem_limit) (GB)	N/A	1,0	N/A	0,5
Límite RAM contenedor MySQL (mem_limit) (GB)	N/A	5,0	N/A	2,5
Asignación de vCPUs para Nginx	4 (todas, sin cuota)	1,5	2 (todas, sin cuota)	0,5
Asignación de vCPUs para MySQL	4 (compartidas con Nginx)	2,5	2 (compartidas con Nginx)	1,5

Los valores derivados de las fórmulas, que son los que se configuraron en la VM, quedan así.

Valor derivado	A Nativo	A Docker	B Nativo	B Docker
procesos_trabajadores (worker_processes)	4	2	2	1
conexiones_por_proceso (worker_connections)	7 204	7 202	7 202	7 201
conexiones_max x costo_por_conexión (MB)	500	500	250	250
memoria_caché de la fórmula (MB)	6 360,8	4 620	2 975,6	2 310
innodb_buffer_pool_size tras el margen (MB)	6 060 (5,92 GB)	4 320 (4,22 GB)	2 724 (2,66 GB)	2 060 (2,01 GB)
Política de swap	swappiness = 1	memswap_limit = mem_limit	swappiness = 1	memswap_limit = mem_limit

Entregable 2: justificación técnica de escalabilidad

Recomiendo separar Nginx y MySQL en dos servidores como primer cambio. Los cálculos muestran que los dos servicios gastan recursos distintos. MySQL es el que consume memoria: en el Escenario A su buffer pool ocupa 5,92 GB de los 8 (74 %) en nativo y 4,22 GB en Docker, y solo las conexiones suman 500 MB. Nginx casi no usa RAM (0,3 GB), pero necesita CPU y descriptores de archivo, con 7 204 conexiones por worker. En un solo servidor, si el tráfico se duplica, las conexiones y el consumo de ambos crecen a la vez sobre un margen de solo 300 MB, y el riesgo es el OOM Killer o la hiperpaginación. La CPU ya es el recurso escaso. En el escenario B, al limitar Nginx a 0,5 vCPU, el rendimiento medido cayó a menos de un tercio (9 714 contra 33 023 solicitudes por segundo, con el cliente corriendo en la misma VM).

Escalar verticalmente alivia, pero deja un único punto de falla y obliga a pagar RAM y CPU para los dos servicios aunque cada uno necesite proporciones distintas. Las réplicas de lectura vienen después. Solo ayudan si las lecturas de la base de datos son el cuello de botella, exigen cambiar la

aplicación para separar lecturas de escrituras y añaden retraso de replicación. Con MySQL en su propio servidor se le puede dar casi toda la RAM, con un buffer pool mayor, y solo si las métricas de lectura lo piden se agrega la réplica.

Resultados de la comprobación en la VM

	A Nativo	A Docker	B Nativo	B Docker
Workers de Nginx (ps / docker top)	4	2	2	1
worker_connections	7 204	7 202	7 202	7 201
Buffer pool de MySQL (MB)	6 060	4 320	2 724	2 060
max_connections	200	200	100	100
RAM usada tras la carga	852 MB (servidor)	514 MiB MySQL + 9,7 MiB Nginx	541 MB (servidor)	359 MiB MySQL + 10,7 MiB Nginx
ab: solicitudes fallidas (de 20 000)	0	0	0	0
ab: solicitudes por segundo	52 337	27 938	33 023	9 714
ab: tiempo medio por solicitud (ms)	7,6	14,3	12,1	41,2
Eventos de OOM en dmesg / OOMKilled	0 / n.a.	0 / false	0 / n.a.	0 / false
Overhead medido del daemon de Docker	n.a.	159 MB	n.a.	144 MB

La prueba de carga solo toca Nginx. No ejercité MySQL con consultas, así que el buffer pool se comprobó como valor configurado y como límite que cabe en la RAM, no bajo estrés. El ab corre en la misma VM y por loopback, por lo que le quita CPU al servidor, y los números sirven para comparar entre vías y no como capacidad absoluta.

Errores clásicos de producción

Error	Cómo lo evita el dimensionado	Evidencia
OOM Killer	Buffer pool, conexiones y margen suman menos que la memoria disponible. En Docker, 4 320 + 500 + 300 = 5 120 MB, igual al límite de 5 GB del contenedor.	dmesg sin eventos de OOM; OOMKilled=false y Restarts=0.
Hiperpaginación (swap thrashing)	En nativo, `vm.swappiness = 1`. En Docker, `memswap_limit` igual a `mem_limit`, de modo que el contenedor no puede usar swap.	memory.swap.max = 0 en los cuatro contenedores; swap usado 0 B.
Too Many Open Files	`worker_rlimit_nofile` 16 384 por encima de `worker_connections` (7 204 como máximo), y `ulimits nofile` de 32 768 y 65 535 en los contenedores.	Max open files 16384 en nativo y 32768 en Docker; ab con 400 conexiones y 0 fallidas.
Contención de CPU	Workers iguales a los núcleos (nativo) o a `techo(cuota)` en Docker, nunca más.	4, 2, 2 y 1 workers en `ps` y `docker top`.

Lista de comprobación

Requisito del caso	Estado	Dónde está
Cálculo para el Escenario A y el B	Hecho	Memoria de cálculo, Entregable 1
Cálculo por la vía nativa (bottom-up) y por Docker (top-down)	Hecho	Memoria de cálculo, pasos 6 a 31
Procesos trabajadores y conexiones por proceso	Hecho	Entregable 1, pasos 8, 14, 22 y 28
Memoria caché y buffer pool de MySQL sin riesgo de OOM	Hecho	Pasos 9, 15, 23 y 29
Overhead de 12,5 %, 15 % y 25 % según escenario y vía	Hecho	Memoria de cálculo
Límites de contenedor (mem_limit y cpus)	Hecho	Pasos 13 y 27
Entregable 1: tabla comparativa	Hecho	Sección Entregable 1
Entregable 2: justificación de escalabilidad (media página)	Hecho	Sección Entregable 2
Errores clásicos analizados	Hecho	Sección Errores clásicos
Capturas de cada paso en la VM	Hecho	Figuras 1 a 37
VM apagada al terminar	Hecho	Última figura

Conclusiones

- La vía nativa deja más memoria útil para MySQL que la de Docker. En el Escenario A el buffer pool es de 5,92 GB contra 4,22 GB, porque el overhead es de 12,5 % y no de 25 %, y porque Nginx no tiene un contenedor de 1 GB reservado aparte.
- En cambio, Docker ofrece límites duros. Cada servicio tiene su tope de memoria y de CPU, y si uno se desborda el OOM Killer lo termina dentro de su cgroup sin llevarse al otro. En nativo, todos comparten la misma RAM y el kernel elige a la víctima.
- Los workers cambian de una vía a otra (4 y 2 en nativo, 2 y 1 en Docker) pero las conexiones por proceso casi no, porque la fórmula está dominada por 1 200 x 4 x 1,5 = 7 200. Lo que cambia es cuántos procesos las reparten, y por eso importa subir `worker_rlimit_nofile`.
- El 25 % de overhead de Docker es una reserva de presupuesto. Lo medido fue de 144 a 159 MB de memoria residente en el daemon, lejos de 1 y 2 GB. Conviene mantener la reserva, porque también cubre la caché de `overlay2` y el kernel, pero no hay que entenderla como lo que ocupa el daemon.
- La cuota de CPU se nota enseguida. Con 0,5 vCPU, Nginx rindió menos de un tercio que en nativo con 2. Antes de reducir la cuota de un contenedor hay que medir su carga.
- El caso se comprobó en una VM con los mismos recursos de cada escenario, y las cuatro configuraciones arrancaron, aceptaron 400 conexiones concurrentes sin fallos y no activaron el OOM Killer. La limitación es que no se estresó MySQL con consultas.

Apagado de las máquinas virtuales

Después de terminar todas las pruebas, apagué la VM con `poweroff` y la devolví a su configuración original (2 048 MB y 2 CPU, la del Vagrantfile) para que los laboratorios anteriores sigan funcionando igual. `VBoxManage list runningvms` no devuelve ninguna máquina. Solo se usó vm-sistemas-operativos; Lab1-Linux no se encendió.

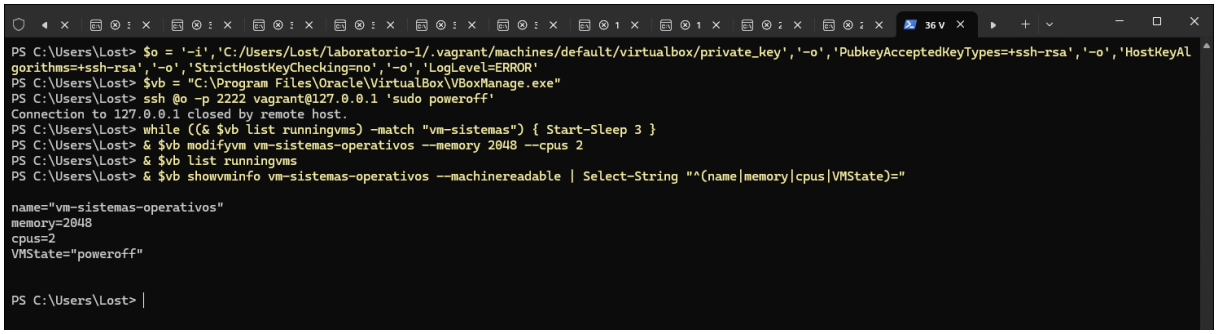


Figura 37. La VM apagada y restaurada a 2 048 MB y 2 CPU; no queda ninguna máquina virtual en ejecución.

Anexos

Anexo A. Comandos ejecutados

Figura 1

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" modifyvm vm-sistemas-operativos --memory 8192 --
cpus 4
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm vm-sistemas-operativos --type headless
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" showvminfo vm-sistemas-operativos --
machinereadable | Select-String "^(name|memory|cpus|VMState)="
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" list runningvms
```

Figura 2

```
cd C:\Users\Lost\Downloads\CasoEstudioSO\entrega\src
cmd /c tree /F
ssh @o -p 2222 vagrant@127.0.0.1 'mkdir -p ~/casoestudio'
scp @o -P 2222 -r A B calc.py vagrant@127.0.0.1:casoestudio/
ssh @o -p 2222 vagrant@127.0.0.1 'ls -R ~/casoestudio'
```

Figura 3

```
whoami; hostname; lsb_release -ds
nproc
free -h
swapon --show
docker --version; docker compose version
sysctl vm.swappiness; ulimit -n
dpkg -l nginx mysql-server apache2-utils 2>&1 | tail -3
docker ps -a; docker images
```

Figura 4

```
python3 calc.py A
python3 calc.py B
```

Figura 5

```
sudo DEBIAN_FRONTEND=noninteractive apt-get update -qq 2>&1 | tail -3
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y nginx mysql-server apache2-utils 2>&1 | tail -14
nginx -v; mysqld --version; ab -V | head -1; systemctl is-active nginx mysql
```

Figura 6

```
cat A/nativo/nginx.conf
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.orig
sudo cp A/nativo/nginx.conf /etc/nginx/nginx.conf
sudo nginx -t
sudo systemctl restart nginx; systemctl is-active nginx
```

Figura 7

```
cat A/nativo/mysql.cnf
sudo cp A/nativo/mysql.cnf /etc/mysql/mysql.conf.d/zz-caso.cnf
sudo systemctl restart mysql; systemctl is-active mysql
sudo systemctl status mysql --no-pager | head -4
```

Figura 8

```
sudo nginx -T 2>/dev/null | grep -E 'worker_processes|worker_connections|worker_rlimit_nofile'
ps -eo pid,ppid,user,cmd | grep '[n]ginx:'
pgrep -fc 'nginx: worker'
grep 'open files' /proc/$(pgrep -f 'nginx: worker' | head -1)/limits
```

Figura 9

```
sudo mysql -e "SELECT @@max_connections AS max_connections, @@innodb_buffer_pool_size/1024/1024 AS
buffer_pool_mb, @@innodb_buffer_pool_size/1024/1024/1024 AS buffer_pool_gb,
@@innodb_buffer_pool_chunk_size/1024/1024 AS chunk_mb;"
sudo mysql -e "SHOW VARIABLES LIKE 'innodb_buffer_pool_%size';"
ps -o pid,rss,vsz,cmd -C mysqld
free -m
```

Figura 10

```
ab -k -n 20000 -c 400 http://127.0.0.1/ 2>&1 | grep -E 'Concurrency Level|Complete requests|Failed
requests|Requests per second|Time per request'
free -m
sudo dmesg | grep -ci -E 'out of memory|killed process'
systemctl is-active nginx mysql
```

Figura 11

```
sudo systemctl stop nginx mysql
systemctl is-active nginx mysql
ss -ltn | grep -E ':80 |:3306 ' | wc -l
cd A/docker
cat docker-compose.yml
```

Figura 12

```
cd A/docker
cat nginx.conf
cat mysql.cnf
```

Figura 13

```
cd A/docker
docker compose pull 2>&1 | tail -6
docker compose up -d 2>&1 | tail -8
docker compose ps
```

Figura 14

```
cd A/docker
docker inspect -f '{{.Name}} Memory={{.HostConfig.Memory}} MemorySwap={{.HostConfig.MemorySwap}}
NanoCpus={{.HostConfig.NanoCpus}} Nofile={{.HostConfig.Ulimits}}' caso_nginx caso_mysql
for c in caso_nginx caso_mysql; do echo $c; docker exec $c cat /sys/fs/cgroup/memory.max
/sys/fs/cgroup/memory.swap.max /sys/fs/cgroup/cpu.max; done
docker stats --no-stream
```

Figura 15

```
docker exec caso_nginx nginx -T 2>/dev/null | grep -E
'worker_processes|worker_connections|worker_rlimit_nofile'
docker top caso_nginx -eo pid,user,cmd
docker exec caso_nginx sh -c 'grep "open files" /proc/$(cat /var/run/nginx.pid)/limits'
```

Figura 16

```
until docker exec caso_mysql mysql -uroot -pcaso2026 -e "SELECT 1" >/dev/null 2>&1; do sleep 2; done
docker exec caso_mysql mysql -uroot -pcaso2026 -e "SELECT @@max_connections AS max_connections,
@@innodb_buffer_pool_size/1024/1024 AS buffer_pool_mb, @@innodb_buffer_pool_size/1024/1024/1024 AS
buffer_pool_gb;" 2>/dev/null
docker stats --no-stream
free -m
```

Figura 17

```
ab -k -n 20000 -c 400 http://127.0.0.1/ 2>&1 | grep -E 'Concurrency Level|Complete requests|Failed
requests|Requests per second|Time per request'
docker stats --no-stream
docker inspect -f '{{.Name}} OOMKilled={{.State.OOMKilled}} Restarts={{.RestartCount}}
Estado={{.State.Status}}' caso_nginx caso_mysql
sudo dmesg | grep -ci -E 'out of memory|killed process'
```

Figura 18

```
ps -o pid,rss,comm -C dockerd,containerd,containerd-shim | sort -k2 -n
ps -C dockerd,containerd,containerd-shim -o rss= | awk '{s+=$1} END {print "Docker + containerd + shims
(MB):", s/1024}'
docker info 2>/dev/null | grep -E "Storage Driver|Cgroup Version|Cgroup Driver"
```

Figura 19

```
cd A/docker
docker compose down 2>&1 | tail -6
docker ps -a
free -m
```

Figura 20

```
ssh @o -p 2222 vagrant@127.0.0.1 'sudo poweroff'
& $vb list runningvms
& $vb modifyvm vm-sistemas-operativos --memory 4096 --cpus 2
& $vb startvm vm-sistemas-operativos --type headless
& $vb showvminfo vm-sistemas-operativos --machinereadable | Select-String
"^(name|memory|cpus|VMState)="
```

Figura 21

```
whoami; hostname; lsb_release -ds
nproc
free -h
swapon --show
sysctl vm.swappiness
docker ps -a
systemctl is-active nginx mysql
```

Figura 22

```
cat B/nativo/nginx.conf
sudo cp B/nativo/nginx.conf /etc/nginx/nginx.conf
sudo nginx -t
sudo systemctl restart nginx; systemctl is-active nginx
```

Figura 23

```
cat B/nativo/mysql.cnf
sudo cp B/nativo/mysql.cnf /etc/mysql/mysql.conf.d/zz-caso.cnf
sudo systemctl restart mysql; systemctl is-active mysql
sudo systemctl status mysql --no-pager | head -4
```

Figura 24

```
sysctl vm.swappiness
sudo tee /etc/sysctl.d/99-caso.conf <<< "vm.swappiness = 1"
sudo sysctl -p /etc/sysctl.d/99-caso.conf
cat /proc/sys/vm/swappiness
swapon --show; free -m
```

Figura 25

```
sudo nginx -T 2>/dev/null | grep -E 'worker_processes|worker_connections|worker_rlimit_nofile'
ps -eo pid,ppid,user,cmd | grep '[n]ginx:'
pgrep -fc 'nginx: worker'
grep 'open files' /proc/$(pgrep -f 'nginx: worker' | head -1)/limits
```

Figura 26

```
sudo mysql -e "SELECT @@max_connections AS max_connections, @@innodb_buffer_pool_size/1024/1024 AS
buffer_pool_mb, @@innodb_buffer_pool_size/1024/1024/1024 AS buffer_pool_gb,
@@innodb_buffer_pool_chunk_size/1024/1024 AS chunk_mb;"
sudo mysql -e "SHOW VARIABLES LIKE 'innodb_buffer_pool_%size';"
ps -o pid,rss,vsz,cmd -C mysqld
free -m
```

Figura 27

```
ab -k -n 20000 -c 400 http://127.0.0.1/ 2>&1 | grep -E 'Concurrency Level|Complete requests|Failed
requests|Requests per second|Time per request'
free -m
sudo dmesg | grep -ci -E 'out of memory|killed process'
systemctl is-active nginx mysql
```

Figura 28

```
sudo systemctl stop nginx mysql
systemctl is-active nginx mysql
ss -ltn | grep -E ':80 |:3306 ' | wc -l
cd B/docker
cat docker-compose.yml
```

Figura 29

```
cd B/docker
cat nginx.conf
cat mysql.cnf
```

Figura 30

```
cd B/docker
docker compose up -d 2>&1 | tail -8
docker compose ps
```

Figura 31

```
cd B/docker
docker inspect -f '{{.Name}} Memory={{.HostConfig.Memory}} MemorySwap={{.HostConfig.MemorySwap}}
NanoCpus={{.HostConfig.NanoCpus}} Nofile={{.HostConfig.Ulimits}}' caso_nginx caso_mysql
for c in caso_nginx caso_mysql; do echo $c; docker exec $c cat /sys/fs/cgroup/memory.max
/sys/fs/cgroup/memory.swap.max /sys/fs/cgroup/cpu.max; done
docker stats --no-stream
```

Figura 32

```
docker exec caso_nginx nginx -T 2>/dev/null | grep -E
'worker_processes|worker_connections|worker_rlimit_nofile'
docker top caso_nginx -eo pid,user,cmd
docker exec caso_nginx sh -c 'grep "open files" /proc/$(cat /var/run/nginx.pid)/limits'
```

Figura 33

```
until docker exec caso_mysql mysql -uroot -pcaso2026 -e "SELECT 1" >/dev/null 2>&1; do sleep 2; done
docker exec caso_mysql mysql -uroot -pcaso2026 -e "SELECT @@max_connections AS max_connections,
@@innodb_buffer_pool_size/1024/1024 AS buffer_pool_mb, @@innodb_buffer_pool_size/1024/1024/1024 AS
buffer_pool_gb;" 2>/dev/null
docker stats --no-stream
free -m
```

Figura 34

```
ab -k -n 20000 -c 400 http://127.0.0.1/ 2>&1 | grep -E 'Concurrency Level|Complete requests|Failed
requests|Requests per second|Time per request'
docker stats --no-stream
docker inspect -f '{{.Name}} OOMKilled={{.State.OOMKilled}} Restarts={{.RestartCount}}
Estado={{.State.Status}}' caso_nginx caso_mysql
sudo dmesg | grep -ci -E 'out of memory|killed process'
```

Figura 35

```
ps -o pid,rss,comm -C dockerd,containerd,containerd-shim | sort -k2 -n
ps -C dockerd,containerd,containerd-shim -o rss= | awk '{s+=$1} END {print "Docker + containerd + shims
(MB):", s/1024}'
docker info 2>/dev/null | grep -E "Storage Driver|Cgroup Version|Cgroup Driver"
```

Figura 36

```
cd B/docker
docker compose down 2>&1 | tail -6
docker ps -a
sudo systemctl disable nginx mysql 2>&1 | tail -4
systemctl is-active nginx mysql; systemctl is-enabled nginx mysql
ss -ltn | grep -E ':80 |:3306 ' | wc -l
```

Figura 37

```
ssh @o -p 2222 vagrant@127.0.0.1 'sudo poweroff'
& $vb modifyvm vm-sistemas-operativos --memory 2048 --cpus 2
& $vb list runningvms
& $vb showvminfo vm-sistemas-operativos --machinereadable | Select-String
"^(name|memory|cpus|VMState)="
```

Anexo B. Archivos del proyecto

calc.py

```
import math, sys

USUARIOS_PICO = 1200
CONEXIONES_USUARIO = 4
MARGEN_CONEXIONES = 1.5
COSTO_CONEXION_MB = 2.5

ESCENARIOS = {
    "A": dict(ram_gb=8, vcpu=4, oh_nativo=0.125, oh_docker=0.25, nginx_nativo_gb=0.30,
              nginx_docker_gb=1.0, nginx_cpu=1.5, max_conn=200, margen_mb=300),
    "B": dict(ram_gb=4, vcpu=2, oh_nativo=0.15, oh_docker=0.25, nginx_nativo_gb=0.25,
              nginx_docker_gb=0.5, nginx_cpu=0.5, max_conn=100, margen_mb=250),
}


def conexiones(workers):
    return int(USUARIOS_PICO * CONEXIONES_USUARIO * MARGEN_CONEXIONES + workers)
```

```
def calcular(e):
    ram_mb = e["ram_gb"] * 1024
    conn_mb = e["max_conn"] * COSTO_CONEXION_MB
    r = {}
    so = ram_mb * e["oh_nativo"]
    cache = ram_mb - so - e["nginx_nativo_gb"] * 1024 - conn_mb
    r["nativo"] = dict(overhead_gb=so / 1024, libre_gb=(ram_mb - so) / 1024, workers=e["vcpu"],
                       conn_proceso=conexiones(e["vcpu"]), conn_mb=conn_mb, cache_mb=cache,
                       buffer_pool_mb=cache - e["margen_mb"])
    so = ram_mb * e["oh_docker"]
    mysql_mb = ram_mb - so - e["nginx_docker_gb"] * 1024
    cache = mysql_mb - conn_mb
    w = math.ceil(e["nginx_cpu"])
    r["docker"] = dict(overhead_gb=so / 1024, libre_gb=(ram_mb - so) / 1024,
nginx_gb=e["nginx_docker_gb"],
                       mysql_gb=mysql_mb / 1024, mysql_mb=mysql_mb, cpu_nginx=e["nginx_cpu"],
                       cpu_mysql=e["vcpu"] - e["nginx_cpu"], workers=w, conn_proceso=conexiones(w),
                       conn_mb=conn_mb, cache_mb=cache, buffer_pool_mb=cache - e["margen_mb"])
    return r


def imprimir(nombre):
    e = ESCENARIOS[nombre]
    r = calcular(e)
    n, d = r["nativo"], r["docker"]
    print("ESCENARIO %s: %d GB RAM, %d vCPU, max_connections=%d, margen=%d MB" % (nombre, e["ram_gb"],
e["vcpu"], e["max_conn"], e["margen_mb"]))
    print("-" * 78)
    print("%-44s %15s %15s" % ("Parametro", "Nativo", "Docker"))
    print("%-44s %15.3f %15.3f" % ("Overhead SO / Docker (GB)", n["overhead_gb"], d["overhead_gb"]))
    print("%-44s %15.3f %15.3f" % ("RAM libre para servicios (GB)", n["libre_gb"], d["libre_gb"]))
    print("%-44s %15s %15.3f" % ("mem_limit Nginx (GB)", "N/A", d["nginx_gb"]))
    print("%-44s %15s %15.3f" % ("mem_limit MySQL (GB)", "N/A", d["mysql_gb"]))
    print("%-44s %15s %15.1f" % ("vCPUs Nginx", "compartidas (%d)" % e["vcpu"], d["cpu_nginx"]))
    print("%-44s %15s %15.1f" % ("vCPUs MySQL", "compartidas (%d)" % e["vcpu"], d["cpu_mysql"]))
    print("%-44s %15d %15d" % ("procesos_trabajadores (worker_processes)", n["workers"], d["workers"]))
    print("%-44s %15d %15d" % ("conexiones_por_proceso (worker_connections)", n["conn_proceso"],
d["conn_proceso"]))
    print("%-44s %15.1f %15.1f" % ("conexiones_max x costo_por_conexion (MB)", n["conn_mb"],
d["conn_mb"]))
    print("%-44s %15.1f %15.1f" % ("memoria_cache (MB)", n["cache_mb"], d["cache_mb"]))
    print("%-44s %15.1f %15.1f" % ("memoria_cache - margen = buffer pool (MB)", n["buffer_pool_mb"],
d["buffer_pool_mb"]))
    print("%-44s %15.2f %15.2f" % ("buffer pool (GB)", n["buffer_pool_mb"] / 1024, d["buffer_pool_mb"]
/ 1024))


if __name__ == "__main__":
    for nombre in sys.argv[1:] or ["A", "B"]:
        imprimir(nombre)
        print()
```

A/nativo/nginx.conf (el del B cambia `worker_processes` a 2 y `worker_connections` a 7202)

```
user www-data;
worker_processes 4;
worker_rlimit_nofile 16384;
pid /run/nginx.pid;

events {
```

```
    worker_connections 7204;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    access_log off;
    keepalive_timeout 65;

    server {
        listen 80 default_server;
        root /var/www/html;
        location / {
            try_files $uri $uri/ =404;
        }
    }
}
```

A/nativo/mysql.cnf (el del B usa `max_connections = 100` e `innodb_buffer_pool_size = 2724M`)

```
[mysqld]
max_connections = 200
innodb_buffer_pool_size = 6060M
innodb_buffer_pool_chunk_size = 4M
innodb_buffer_pool_instances = 1
```

A/docker/docker-compose.yml (el del B usa `512m` y `0.5` para Nginx, y `2560m` y `1.5` para MySQL)

```
services:
  nginx:
    image: nginx:1.25
    container_name: caso_nginx
    mem_limit: 1g
    memswap_limit: 1g
    cpus: 1.5
    ulimits:
      nofile:
        soft: 32768
        hard: 32768
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro

  mysql:
    image: mysql:5.7
    container_name: caso_mysql
    mem_limit: 5g
    memswap_limit: 5g
    cpus: 2.5
    ulimits:
      nofile:
        soft: 65535
        hard: 65535
    environment:
      MYSQL_ROOT_PASSWORD: caso2026
    volumes:
      - ./mysql.cnf:/etc/mysql/conf.d/caso.cnf:ro
```

A/docker/nginx.conf y **A/docker/mysql.cnf** (en el B: 1 worker, 7201 conexiones y buffer pool de 2060M)

```
user nginx;
worker_processes 2;
worker_rlimit_nofile 16384;
pid /var/run/nginx.pid;

events {
    worker_connections 7202;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    access_log off;
    keepalive_timeout 65;

    server {
        listen 80 default_server;
        root /usr/share/nginx/html;
        location / {
            try_files $uri $uri/ =404;
        }
    }
}
```

```
[mysqld]
max_connections = 200
innodb_buffer_pool_size = 4320M
innodb_buffer_pool_chunk_size = 4M
innodb_buffer_pool_instances = 1
```